

Codeforces 449B

Jzzhu and Cities

Dijkstra com análise de arestas redundantes

Grupo F

Integrantes: João Vitor Silva • Antonio Davi • Pablo Dornelles

Orientador: Prof. Me Ricardo Carubbi

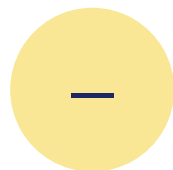
Linguagem: Python 3 • Plataforma: Codeforces

1. O problema



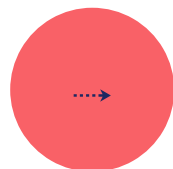
n

idades
(capital = 1)



m

estradas
bidirecionais



k

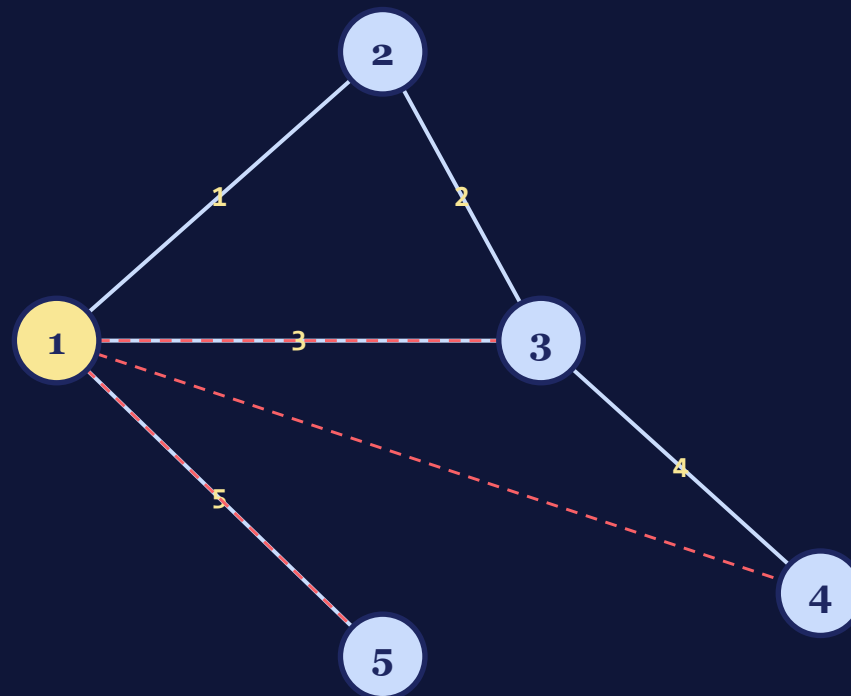
trens
da capital

Objetivo: fechar o máximo de trens sem aumentar a distância da capital a nenhuma cidade.



trens podem ser fechados (saída do exemplo 1)

Exemplo 1



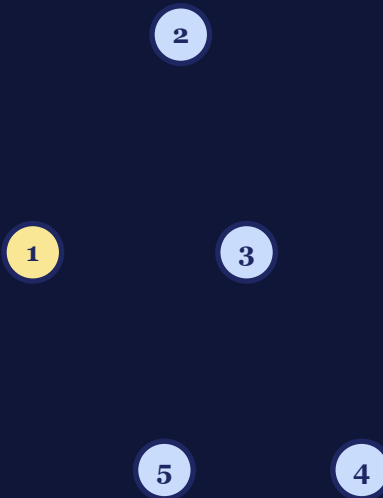
— estrada

- - - trem (da capital)

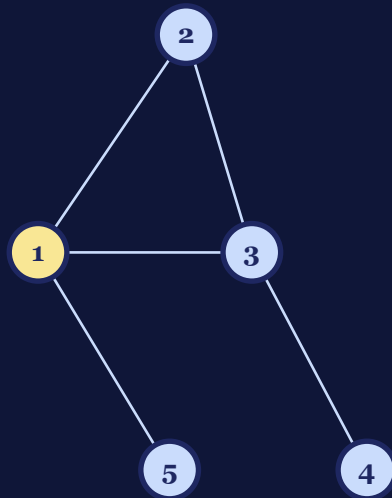
2. Modelagem como grafo

Cada linha da entrada vira um elemento do grafo — em três etapas:

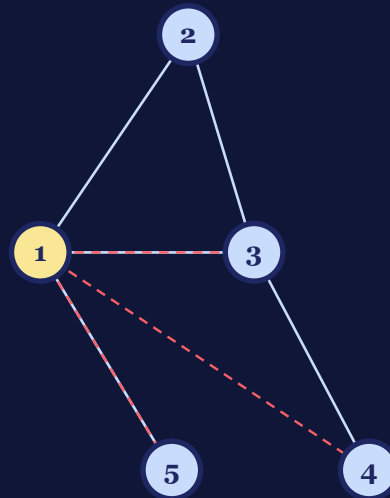
Cidades → vértices



Estradas → arestas



Trens → arestas extras



Por que Dijkstra?

- ≥ 0 Pesos não negativos ($x, y \geq 1$)
- 1 Origem única: a capital
- +
- Trens são só arestas extras

Representação

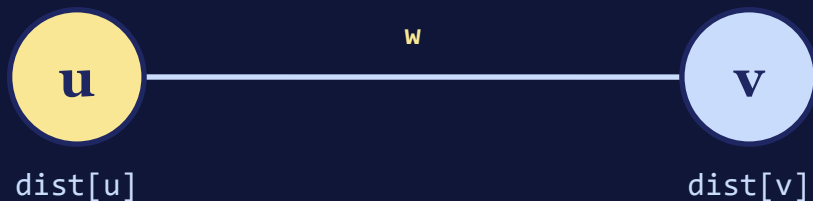
`adj[u] = [ (vizinho, peso, é_trem) ]`

lista de adjacência • $V = n$ • $E = 2m + k$

3. Estratégia: Dijkstra + via_estrada

Variação: uma única execução de Dijkstra, com uma anotação extra por vértice — sem expansão de estado.

Relaxar $u \rightarrow v$



$$nd = dist[u] + w$$

$nd < dist[v] \rightarrow dist[v]=nd, via_estrada[v] = (\text{não é trem})$

$nd = dist[v] \text{ e } estrada \rightarrow via_estrada[v] = True$



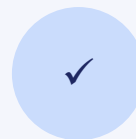
Fila de prioridade

heapq sempre extrai o vértice de menor distância.



Inicialização

$dist[1]=0$, demaiss ∞ . $via_estrada[1]=True$.

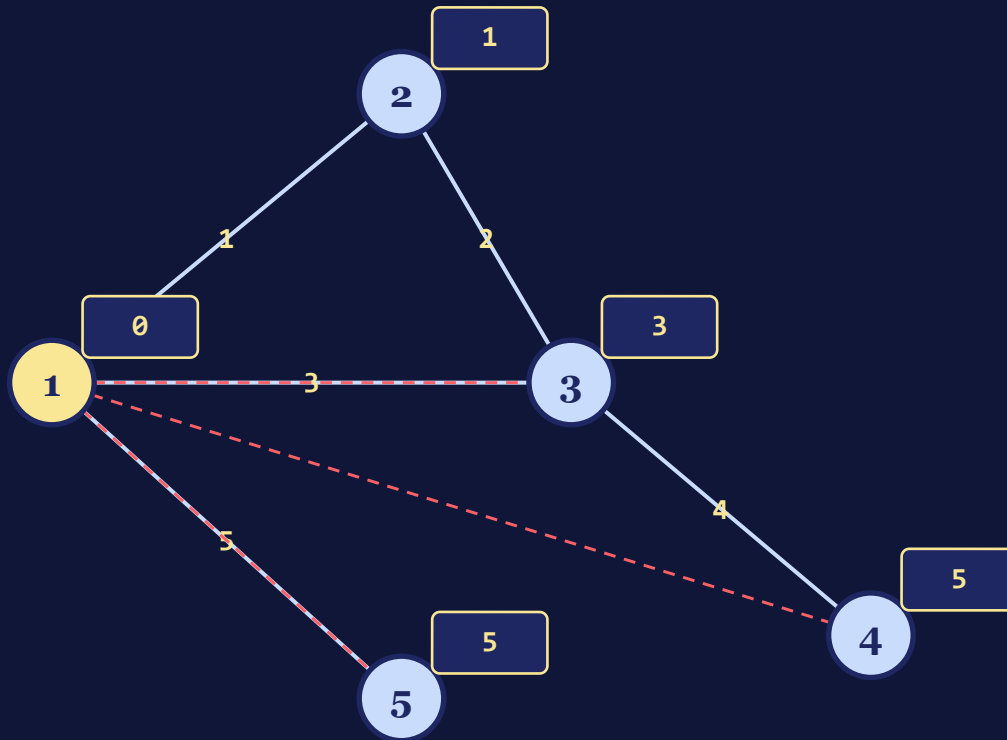


Parada

Fila vazia $\rightarrow dist[]$ final, com trens já incluídos.

4. Dijkstra em ação — Exemplo 1

Distâncias finais (pílulas douradas)



Vetores ao final

índice	1	2	3	4	5
dist	0	1	3	5	5
via_estrada	T	T	T	F	T

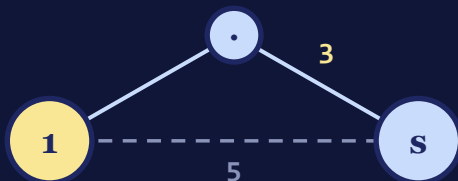
Como ler

- Cidades 2, 3, 5 têm caminho ótimo por estradas.
- Só a cidade 4 depende do trem (via_estrada = F).
- Logo, no exemplo, 2 dos 3 trens podem ser fechados.

5. Quando fechar cada trem?

Trem inútil

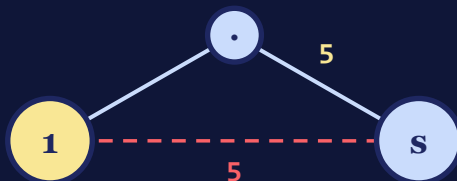
$y > \text{dist}[s]$



× FECHA

Alternativa por estrada

$y = \text{dist}[s]$, via estrada



× FECHA

Trem essencial

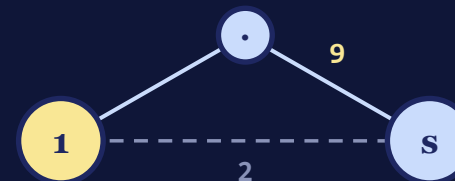
$y = \text{dist}[s]$, sem estrada



✓ MANTÉM 1

Caso impossível

$y < \text{dist}[s]$



— não ocorre

No exemplo 1

trem (3,5): $5 > 3 \rightarrow$ fecha

trem (4,5): $5 = 5$, sem estrada $\rightarrow$ mantém

trem (5,5): $5 = 5$, há estrada $\rightarrow$ fecha

6. Complexidade e casos especiais

Tempo

$$O((V + E) \cdot \log V)$$

$V = n$ *vértices*

$E = 2m + k$ *estradas $\times 2$ + trens*

$\log V$ *heap (heapq)*

1625 ms

tempo (lim. 2 s)

180 MB

memória (lim. 256 MB)

Casos especiais

10⁹

Pesos grandes

soma $\sim 10^{14}$ — int do Python / long em Java.

=

Empate trem $\times$ estrada

via_estrada detecta e fecha o trem.

≡

Trens repetidos

agrupa por destino; mantém só o necessário.

↓

Trem melhora distância

é só mais uma aresta no Dijkstra.

Resultado

Accepted

#376408159

submissão

1625 ms

tempo (limite 2 s)

180 MB

memória (limite 256 MB)

Em resumo

- Trens são apenas arestas extras a partir da capital.
- Um único Dijkstra + a marca `via_estrada` já resolve.
- Conta-se direto, por destino, quantos trens podem fechar.